

Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Научный руководитель

_____ Горностаев Александр
Сергеевич

«__» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель
образовательной программы

_____ Син Денис
Дмитриевич

«__» _____ 2026 г.

LISTLY BACKEND

Программа и методика испытаний

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.05.01 51 01-1-ЛУ

Исполнители:

Студент группы БДРИП242

_____ / Магомедов Махач К. /

«__» _____ 2026 г.

Инва.№ подп	Подп. и дата	Взам. инв.№	Инва.№ дубл.	Подп. и дата

УТВЕРЖДЕН

RU.17701729.05.01 51 01-1-ЛЮ

LISTLY BACKEND

Программа и методика испытаний

RU.17701729.05.01 51 01-1

Листов 19

Инов.№ подп	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Программа и методика испытаний — это документ, в котором содержится информация о программном продукте, а также полное описание приемочных испытаний для данного программного продукта.

Настоящая Программа и методика испытаний для «ListLy Backend» содержит следующие разделы: «Объект испытаний», «Цель испытаний», «Требования к программе», «Требования к программной документации», «Средства и порядок испытаний», «Методы испытаний», «Список используемой литературы».

В разделе «Объект испытаний» указано наименование, краткая характеристика и назначение программы.

В разделе «Цель испытаний» указана цель проведения испытаний. Раздел «Требования к программе» содержит основные требования к программе, которые подлежат проверке во время испытаний (требования к функционалу и интерфейсу).

Раздел «Требования к программным документам» содержит состав программной документации, которая представляется на испытания.

Раздел «Средства и порядок испытаний» содержит информацию о технических и программных средствах, которые следует использовать во время испытаний, а также порядок этих испытаний.

Раздел «Методы испытаний» содержит информацию об используемых методах испытаний.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
8. ГОСТ 19.301-79 [8]: Программа и методика испытаний. Требования к содержанию и оформлению.

Изменения к данной Программе и методике испытаний оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СОДЕРЖАНИЕ

1. ОБЪЕКТ ИСПЫТАНИЙ	4
1.1. Наименование программы	4
1.2. Краткая характеристика области применения программы	4
2. ЦЕЛЬ ИСПЫТАНИЙ	5
3. ТРЕБОВАНИЯ К ПРОГРАММЕ	6
3.1. Требования к функциональным характеристикам	6
3.1.1. Состав выполняемых функций	6
3.2. Требования к входным и выходным данным	6
3.2.1. Организация входных данных	6
3.2.2. Организация выходных данных	7
3.3. Требования к интерфейсу	7
4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ	9
5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ	10
5.1. Технические средства	10
5.2. Программные средства	10
5.3. Порядок проведения испытаний	10
5.4. Требования к персоналу	11
6. МЕТОДЫ ИСПЫТАНИЙ	12
6.1. Общие положения	12
6.2. Проверка регистрации пользователя	12
6.3. Проверка входа пользователя и получения JWT	12
6.4. Проверка защищенного маршрута без JWT	13
6.5. Проверка создания медиа-элемента администратором	13
6.6. Проверка запрета создания медиа-элемента обычным пользователем	13
6.7. Проверка получения глобального каталога	13
6.8. Проверка поиска медиа через Meilisearch	14
6.9. Проверка стартовой выдачи каталога для вкладки поиска	14
6.10. Проверка добавления медиа-элемента в пользовательскую коллекцию	14
6.11. Проверка изменения статуса, оценки, заметки и избранного	14
6.12. Проверка создания, переименования и удаления пользовательской папки	15
6.13. Проверка удаления ссылок на папку из пользовательских записей	15
6.14. Проверка переиндексации поискового индекса администратором	15
6.15. Проверка защищенного маршрута с некорректным JWT	15
6.16. Проверка обращения к несуществующему медиа-элементу	16
6.17. Проверка некорректных параметров пагинации	16
6.18. Проверка пустого поискового запроса	16
6.19. Итоговые результаты испытаний	16
СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ	18

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

1. ОБЪЕКТ ИСПЫТАНИЙ

1.1. Наименование программы

Наименование программы: «ListLy Backend».

Наименование программы на английском языке: «ListLy Backend».

1.2. Краткая характеристика области применения программы

ListLy Backend — backend-система для организации пользовательского медиaproстранства. Система предназначена для хранения и управления пользовательскими медиакolleкциями, включая фильмы, сериалы и игры. Пользователь может сохранять медиа-элементы, устанавливать статусы, оценки, заметки, добавлять элементы в пользовательские папки и выполнять поиск по глобальному каталогу. Backend предоставляет REST API, выполняет аутентификацию и авторизацию через JWT, разграничивает права доступа по ролям USER и ADMIN, взаимодействует с MongoDB как основным хранилищем данных и использует Meilisearch для полнотекстового поиска. Развертывание проекта выполняется в Docker Compose совместно с MongoDB и Meilisearch.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

2. ЦЕЛЬ ИСПЫТАНИЙ

Целью испытаний является проверка корректности выполнения программой функций, изложенных в п. 4 «Требования к программе» документа «Техническое задание» из комплекта документации в соответствии с ЕСПД (Единой системой программной документации).

Для backend-системы ListLy критичными являются следующие свойства:

- корректность работы REST API и обработки HTTP-запросов;
- надежность механизмов аутентификации и авторизации через JWT;
- соблюдение разграничения прав доступа между ролями USER и ADMIN;
- целостность данных при операциях с пользовательскими коллекциями и папками;
- согласованность данных между MongoDB и поисковым индексом Meilisearch;
- корректность обработки ошибок и исключительных ситуаций;
- соответствие временным характеристикам для интерактивной работы.

Испытания должны подтвердить соответствие системы требованиям к:

- функциональности управления медиаколлекциями;
- работе с глобальным каталогом;
- полнотекстовому поиску;
- пользовательским папкам;
- административным функциям;
- взаимодействию с MongoDB и Meilisearch.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

3. ТРЕБОВАНИЯ К ПРОГРАММЕ

3.1. Требования к функциональным характеристикам

3.1.1. Состав выполняемых функций

Программа должна соответствовать следующим функциональным требованиям, указанным в документе «ListLy Backend. Техническое задание»

1. Функции управления доступом:
 - регистрация пользователей через /auth/register
 - аутентификация через /auth/login с выдачей JWT-токена
 - разграничение прав по ролям USER и ADMIN
2. Функции работы с глобальным каталогом:
 - CRUD операции для медиа-элементов (только для ADMIN)
 - получение информации о медиа-элементах
3. Функции поиска:
 - полнотекстовый поиск по каталогу через Meilisearch
 - получение стартовой подборки каталога без поисковой строки через /media/discover
 - ограничение и смещение результатов
4. Функции пользовательской коллекции:
 - добавление/удаление медиа в коллекцию
 - управление статусами, оценками и заметками
 - фильтрация коллекции по различным параметрам
5. Функции работы с папками:
 - создание/переименование/удаление папок
 - добавление медиа в папки
 - автоматическая очистка ссылок при удалении папок
6. Сервисные функции:
 - проверка работоспособности компонентов
 - обработка ошибок и формирование структурированных ответов
 - административная переиндексация поискового индекса

3.2. Требования к входным и выходным данным

3.2.1. Организация входных данных

1. Данные аутентификации:
 - логин/пароль при регистрации
 - JWT-токен для авторизованных запросов
2. Данные медиа-элементов:
 - обязательные поля: название, тип, статус
 - дополнительные метаданные: описание, жанры, постер
3. Параметры поиска:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- поисковая строка
- параметр ограничения количества результатов `limit`
- параметр смещения результатов `offset`

4. Данные пользовательской коллекции:

- идентификатор медиа-элемента
- статус пользовательской коллекции (PLANNED, IN_PROGRESS, COMPLETED, DROPPED; планирую/в процессе/завершено/брошено)
- пользовательская оценка (0-10)
- текст заметки
- список папок

Все входные данные передаются в формате JSON через HTTP-запросы

3.2.2. Организация выходных данных

1. Успешные ответы:

- данные аутентификации (JWT-токен)
- списки медиа-элементов и пользовательских записей
- результаты поиска с учетом `limit` и `offset`
- информация о папках пользователя
- статус работы сервисов

2. Ответы об ошибках:

- структурированные сообщения об ошибке и HTTP-статус
- стандартные HTTP-статусы (400, 401, 403, 404, 409, 500, 502, 503)
- разделение ошибок по типам (валидация, доступ, данные)

3. Формат данных:

- JSON с UTF-8 кодировкой
- стабильная структура данных для успешных ответов
- поле `error` для ответов с ошибками

3.3. Требования к интерфейсу

1. REST API:

- стандартные HTTP-методы (GET, POST, PATCH, DELETE)
- ресурсо-ориентированные URL (`/media`, `/user-media`, `/folders`)
- группировка маршрутов по функциональным областям (аутентификация, глобальный каталог, поиск, пользовательская коллекция, папки)

2. Аутентификация:

- передача JWT в заголовке `Authorization`
- обязательная аутентификация для защищенных `endpoints`

3. Формат запросов/ответов:

- `Content-Type: application/json`
- валидация входных данных

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- стандартизированные коды ошибок

4. Документирование:

- описание всех endpoints
- примеры запросов и ответов
- коды возможных ошибок

Описание основных endpoints приводится в программной документации и проверяется в рамках сценариев испытаний. Примеры запросов и ответов могут быть представлены в составе материалов тестирования.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

4. ТРЕБОВАНИЯ К ПРОГРАММНОЙ ДОКУМЕНТАЦИИ

На испытание должна быть представлена документация в следующем составе:

1. «ListLy Backend». Техническое задание (ГОСТ 19.201-78 [7]).
2. «ListLy Backend». Пояснительная записка (ГОСТ 19.404-79 [10]).
3. «ListLy Backend». Программа и методика испытаний (ГОСТ 19.301-79 [8]).
4. «ListLy Backend». Текст программы (ГОСТ 19.401-78 [9]).
5. «ListLy Backend». Руководство оператора (ГОСТ 19.505-79 [11]).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

5. СРЕДСТВА И ПОРЯДОК ИСПЫТАНИЙ

5.1. Технические средства

Для проведения испытаний ListLy Backend требуется следующее оборудование:

- Сервер или рабочая станция с процессором не менее 2 ядер и 4 ГБ оперативной памяти;
- SSD-накопитель с 10 ГБ свободного пространства для хранения данных;
- Сетевое подключение для взаимодействия между компонентами системы.

Минимальные рекомендуемые характеристики:

- 4-ядерный процессор;
- 8 ГБ оперативной памяти;
- 20 ГБ свободного дискового пространства.

5.2. Программные средства

Для проведения испытаний необходимо следующее программное обеспечение:

- Docker Engine версии 20.10+;
- Docker Compose версии 2.0+;
- MongoDB 7.x в контейнере;
- Meilisearch 1.x в контейнере;
- JRE 21 при запуске backend-приложения вне контейнера;
- Gradle Wrapper проекта для запуска автоматизированных проверок;
- JUnit 5, MockK и Ktor testApplication для модульных и маршрутных тестов;
- Postman, curl или аналогичный инструмент для ручной проверки REST API;
- Утилиты для мониторинга сети и журналирования.

5.3. Порядок проведения испытаний

Испытания проводятся в следующей последовательности:

1. Запуск автоматизированных проверок backend-кода:
 - Переход в каталог проекта backend;
 - Выполнение команды `./gradlew test --console=plain`;
 - Анализ результата выполнения модульных, маршрутных и интеграционных по уровню взаимодействия тестов.
2. Развертывание тестового стенда с помощью Docker Compose:
 - Запуск контейнеров MongoDB и Meilisearch;
 - Запуск контейнера с backend-приложением.
3. Подготовка тестовых данных:
 - Создание тестовых пользователей с ролями USER и ADMIN;
 - Заполнение глобального каталога тестовыми медиа-элементами.
4. Выполнение ручных функциональных сценариев через Postman, curl или аналогичный HTTP-клиент:
 - Проверка аутентификации и авторизации;
 - Тестирование операций с глобальным каталогом;
 - Проверка работы пользовательских коллекций и папок;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

- Тестирование полнотекстового поиска;
- Проверка обработки ошибок и исключительных ситуаций.

5. Сбор и анализ результатов тестирования.

5.4. Требования к персоналу

Для проведения испытаний требуется специалист со следующими навыками:

- Базовые знания работы с Docker и Docker Compose;
- Опыт тестирования REST API;
- Понимание принципов работы MongoDB и JWT-аутентификации;
- Умение анализировать логи и диагностировать проблемы.

Дополнительно рекомендуется знание:

- Основ Kotlin и Ktor;
- Принципов работы поисковых систем (Meilisearch);
- Методологий тестирования backend-систем.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6. МЕТОДЫ ИСПЫТАНИЙ

6.1. Общие положения

Испытания проводятся методом автоматизированной и ручной функциональной проверки программы ListLy Backend. Автоматизированные проверки выполняются средствами Gradle, JUnit 5, MockK и Ktor testApplication. Ручные сценарии REST API выполняются на развернутом тестовом стенде с backend-приложением, MongoDB и Meilisearch с использованием Postman, curl или аналогичного инструмента отправки HTTP-запросов.

Во всех сценариях проверяются HTTP-статусы, структура JSON-ответов, корректность обработки входных данных и соблюдение ограничений доступа по ролям USER и ADMIN. Для защищенных маршрутов JWT-токен передается в заголовке Authorization.

Автоматизированные тесты разделены по уровню проверки. Модульные тесты сервисов проверяют бизнес-логику регистрации, управления глобальным каталогом, пользовательскими коллекциями, папками и поиском без запуска внешних сервисов. Маршрутные тесты Ktor testApplication проверяют HTTP-слой, статусы ответов, обработку ошибок и разграничение доступа. Тесты репозитория Meilisearch используют локальный mock HTTP server и проверяют корректность HTTP-запросов к поисковому сервису без обращения к внешней сети.

Для запуска автоматизированных проверок используется команда:

```
./gradlew test --console=plain
```

Успешным результатом считается завершение задачи test без ошибок и падений тестов.

6.2. Проверка регистрации пользователя

Цель проверки: убедиться, что система создает нового пользователя с ролью USER.

Предварительные условия: backend-приложение запущено, MongoDB доступна, пользователь с указанным логином отсутствует.

Входные данные: JSON-запрос с логином и паролем пользователя.

Порядок действий: отправить POST-запрос на /auth/register; проверить HTTP-статус и тело ответа; повторно выполнить регистрацию с тем же логином для проверки обработки дубликата.

Ожидаемый результат: при корректных данных пользователь создается, пароль не хранится в открытом виде; при повторной регистрации возвращается сообщение об ошибке с соответствующим HTTP-статусом.

6.3. Проверка входа пользователя и получения JWT

Цель проверки: убедиться, что зарегистрированный пользователь может выполнить вход и получить JWT-токен.

Предварительные условия: в системе существует пользователь с известными учетными данными.

Входные данные: JSON-запрос с логином и паролем пользователя.

Порядок действий: отправить POST-запрос на /auth/login; проверить HTTP-статус; сохранить полученный токен; выполнить вход с неверным паролем.

Ожидаемый результат: при корректных учетных данных возвращается JWT-токен; при неверных данных вход отклоняется, токен не выдается.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

6.4. Проверка защищенного маршрута без JWT

Цель проверки: убедиться, что защищенные маршруты недоступны без токена.

Предварительные условия: backend-приложение запущено, в системе есть защищенные маршруты пользовательской коллекции или папок.

Входные данные: HTTP-запрос без заголовка Authorization.

Порядок действий: отправить запрос к защищенному маршруту, например GET /user-media или GET /folders, без JWT-токена.

Ожидаемый результат: сервер возвращает HTTP-статус 401 Unauthorized, прикладная операция не выполняется.

6.5. Проверка создания медиа-элемента администратором

Цель проверки: убедиться, что пользователь с ролью ADMIN может создать запись глобального каталога.

Предварительные условия: существует пользователь ADMIN, выполнен вход и получен JWT-токен администратора.

Входные данные: JSON-запрос с названием, описанием, типом, статусом и жанрами медиа-элемента.

Порядок действий: отправить POST-запрос на /media с JWT администратора; проверить ответ; выполнить запрос получения созданного элемента из глобального каталога.

Ожидаемый результат: медиа-элемент создается в MongoDB, возвращается успешный HTTP-статус и данные созданной записи.

6.6. Проверка запрета создания медиа-элемента обычным пользователем

Цель проверки: убедиться, что пользователь с ролью USER не может выполнять административные операции с глобальным каталогом.

Предварительные условия: существует пользователь USER, выполнен вход и получен JWT-токен пользователя.

Входные данные: JSON-запрос с данными медиа-элемента.

Порядок действий: отправить POST-запрос на /media с JWT обычного пользователя.

Ожидаемый результат: сервер возвращает HTTP-статус 403 Forbidden или иной установленный статус отказа в доступе; медиа-элемент не создается.

6.7. Проверка получения глобального каталога

Цель проверки: убедиться, что клиент может получить сведения о медиа-элементах глобального каталога.

Предварительные условия: в MongoDB есть один или несколько медиа-элементов глобального каталога.

Входные данные: HTTP-запрос к маршруту получения медиа-элементов или конкретного элемента.

Порядок действий: отправить GET-запрос на /media/{mediaId} либо другой маршрут чтения каталога; проверить HTTP-статус и структуру ответа.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Ожидаемый результат: сервер возвращает HTTP-статус 200 и JSON с данными медиа-элемента, включая идентификатор, название, тип и статус.

6.8. Проверка поиска медиа через Meilisearch

Цель проверки: убедиться, что поиск по каталогу выполняется через поисковый индекс Meilisearch.

Предварительные условия: Meilisearch запущен, индекс содержит данные глобального каталога.

Входные данные: поисковая строка, параметры limit и offset.

Порядок действий: отправить GET-запрос на /media/search с параметрами поиска; проверить HTTP-статус и список результатов с учетом limit и offset.

Ожидаемый результат: сервер возвращает HTTP-статус 200 и JSON со списком найденных медиа-элементов; результаты соответствуют поисковой строке.

6.9. Проверка стартовой выдачи каталога для вкладки поиска

Цель проверки: убедиться, что маршрут /media/discover возвращает стартовую подборку медиа без поисковой строки.

Предварительные условия: в MongoDB есть один или несколько медиа-элементов глобального каталога.

Входные данные: параметры limit и offset (или их значения по умолчанию).

Порядок действий: отправить GET-запрос на /media/discover; проверить HTTP-статус и состав ответа; повторить запрос с явными параметрами пагинации.

Ожидаемый результат: сервер возвращает HTTP-статус 200 и JSON со списком медиа-элементов; при некорректных значениях пагинации возвращается ошибка валидации.

6.10. Проверка добавления медиа-элемента в пользовательскую коллекцию

Цель проверки: убедиться, что авторизованный пользователь может добавить элемент глобального каталога в свою коллекцию.

Предварительные условия: пользователь USER авторизован, в глобальном каталоге существует медиа-элемент.

Входные данные: JSON-запрос с идентификатором медиа-элемента и начальными атрибутами пользовательской записи.

Порядок действий: отправить POST-запрос на /user-media с JWT пользователя; затем запросить список пользовательской коллекции.

Ожидаемый результат: запись создается в коллекции пользователя, содержит ссылку на медиа-элемент и возвращается в списке пользовательских записей.

6.11. Проверка изменения статуса, оценки, заметки и избранного

Цель проверки: убедиться, что пользователь может изменять атрибуты собственной записи коллекции.

Предварительные условия: пользователь авторизован, в его коллекции существует запись user-media.

Входные данные: JSON-запросы со статусом коллекции, пользовательской оценкой, заметкой и признаком избранного.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Порядок действий: отправить PATCH-запрос на `/user-media/{userMediaId}` для изменения оценки и заметки; отправить PATCH-запрос на `/user-media/{userMediaId}/status`; отправить PATCH-запрос на `/user-media/{userMediaId}/favourite`; повторно получить запись.

Ожидаемый результат: измененные поля сохраняются в MongoDB и возвращаются в ответе API; пользователь не может изменить чужую запись.

6.12. Проверка создания, переименования и удаления пользовательской папки

Цель проверки: убедиться, что пользователь может управлять собственными папками.

Предварительные условия: пользователь USER авторизован.

Входные данные: JSON-запрос с названием папки, затем JSON-запрос с новым названием.

Порядок действий: отправить POST-запрос на `/folders`; отправить PATCH-запрос на `/folders/{folderId}`; отправить DELETE-запрос на `/folders/{folderId}`; получить список папок пользователя.

Ожидаемый результат: папка создается, переименовывается и удаляется только в пределах данных текущего пользователя; после удаления папка отсутствует в списке.

6.13. Проверка удаления ссылок на папку из пользовательских записей

Цель проверки: убедиться, что при удалении папки ссылки на нее удаляются из пользовательских записей коллекции.

Предварительные условия: пользователь авторизован, создана папка, существует пользовательская запись коллекции, связанная с этой папкой.

Входные данные: идентификатор папки и идентификатор пользовательской записи.

Порядок действий: добавить папку к записи через PATCH-запрос на `/user-media/{userMediaId}/folders`; удалить папку через DELETE-запрос на `/folders/{folderId}`; повторно получить пользовательскую запись.

Ожидаемый результат: папка удаляется, ее идентификатор отсутствует в списке `folderIds` связанных пользовательских записей.

6.14. Проверка переиндексации поискового индекса администратором

Цель проверки: убедиться, что пользователь с ролью ADMIN может запустить переиндексацию Meilisearch.

Предварительные условия: существует пользователь ADMIN, MongoDB содержит медиа-элементы глобального каталога, Meilisearch доступен.

Входные данные: JWT-токен администратора.

Порядок действий: отправить POST-запрос на `/media/admin/reindex`; проверить успешный HTTP-статус; выполнить поисковый запрос по одному из элементов каталога.

Ожидаемый результат: сервер выполняет переиндексацию, поисковый индекс синхронизируется с данными MongoDB, поиск возвращает актуальные результаты.

6.15. Проверка защищенного маршрута с некорректным JWT

Цель проверки: убедиться, что защищенные маршруты отклоняют запросы с некорректным JWT-токеном.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

Предварительные условия: backend-приложение запущено, в системе есть защищенные маршруты пользовательской коллекции или папок.

Входные данные: HTTP-запрос с заголовком `Authorization: Bearer <некорректный-токен>`.

Порядок действий: отправить запрос к защищенному маршруту, например `GET /user-media`, с некорректным JWT-токеном; проверить HTTP-статус и отсутствие выполнения прикладной операции.

Ожидаемый результат: сервер возвращает HTTP-статус 401 Unauthorized, данные пользователя не возвращаются и операция не выполняется.

6.16. Проверка обращения к несуществующему медиа-элементу

Цель проверки: убедиться, что сервер корректно обрабатывает запрос к отсутствующему элементу глобального каталога.

Предварительные условия: backend-приложение запущено, MongoDB доступна, в глобальном каталоге отсутствует медиа-элемент с указанным идентификатором.

Входные данные: идентификатор несуществующего медиа-элемента.

Порядок действий: отправить GET-запрос на `/media/{mediaId}` с несуществующим `mediaId`; проверить HTTP-статус и тело ответа.

Ожидаемый результат: сервер возвращает HTTP-статус 404 Not Found, данные медиа-элемента не возвращаются.

6.17. Проверка некорректных параметров пагинации

Цель проверки: убедиться, что сервер отклоняет некорректные значения параметров `limit` и `offset`.

Предварительные условия: backend-приложение запущено, MongoDB и Meilisearch доступны.

Входные данные: отрицательное значение `offset` или недопустимое значение `limit`.

Порядок действий: отправить GET-запрос на `/media/discover?limit=-1&offset=0` или `/media/search?query=test&limit=0&offset=-1`; проверить HTTP-статус и сообщение об ошибке.

Ожидаемый результат: сервер возвращает HTTP-статус 400 Bad Request, операция выборки не выполняется с некорректными параметрами.

6.18. Проверка пустого поискового запроса

Цель проверки: убедиться, что пустая поисковая строка обрабатывается предсказуемо и не приводит к ошибке сервера.

Предварительные условия: backend-приложение запущено, Meilisearch доступен.

Входные данные: пустое значение параметра `query`.

Порядок действий: отправить GET-запрос на `/media/search?query=&limit=12&offset=0`; проверить HTTP-статус и структуру JSON-ответа.

Ожидаемый результат: сервер возвращает HTTP-статус 200 OK и пустой список результатов поиска.

6.19. Итоговые результаты испытаний

По результатам выполнения автоматизированных, функциональных и негативных сценариев испытаний проверяемые функции backend-системы ListLy признаны работоспособными в

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

пределах заявленных условий испытаний. Автоматизированные проверки выполнялись командой `./gradlew test --console=plain`; результат выполнения: 94 теста, 0 ошибок, 0 падений, 0 пропущенных тестов. Результаты относятся к версии исходного кода, представленной в составе комплекта проекта на момент проведения испытаний.

№	Наименование проверки	Результат
1	Проверка регистрации пользователя	Успешно
2	Проверка входа пользователя и получения JWT	Успешно
3	Проверка защищенного маршрута без JWT	Успешно
4	Проверка создания медиа-элемента администратором	Успешно
5	Проверка запрета создания медиа-элемента обычным пользователем	Успешно
6	Проверка получения глобального каталога	Успешно
7	Проверка поиска медиа через Meilisearch	Успешно
8	Проверка стартовой выдачи каталога для вкладки поиска	Успешно
9	Проверка добавления медиа-элемента в пользовательскую коллекцию	Успешно
10	Проверка изменения статуса, оценки, заметки и избранного	Успешно
11	Проверка создания, переименования и удаления пользовательской папки	Успешно
12	Проверка удаления ссылок на папку из пользовательских записей	Успешно
13	Проверка переиндексации поискового индекса администратором	Успешно
14	Проверка защищенного маршрута с некорректным JWT	Успешно
15	Проверка обращения к несуществующему медиа-элементу	Успешно
16	Проверка некорректных параметров пагинации	Успешно
17	Проверка пустого поискового запроса	Успешно

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
14. Kotlin Documentation. URL: <https://kotlinlang.org/docs/home.html>
15. Ktor Documentation. URL: <https://ktor.io/docs/>
16. MongoDB Documentation. URL: <https://www.mongodb.com/docs/>
17. Meilisearch Documentation. URL: <https://www.meilisearch.com/docs>
18. Docker Documentation. URL: <https://docs.docker.com/>
19. Gradle User Manual. URL: <https://docs.gradle.org/current/userguide/userguide.html>
20. KMongo Documentation. URL: <https://litote.org/kmongo/>
21. JSON Web Token (JWT), RFC 7519. URL: <https://www.rfc-editor.org/rfc/rfc7519>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.05.01 51 01-1				
Инв. № подл.	Подп. и дата	Взам. Инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]